

# 深圳大学实验报告

课程名称： 算法设计与分析

实验项目名称： 动态规划—鸡蛋掉落问题

学院： 计算机与软件学院

专业： 计算机科学与技术

指导教师： 刘刚

报告人： 汉雨 学号： 2024040042

实验时间： 2026年5月24日—2026年5月 日

实验报告提交时间： 2026年5月 日

教务部制

表 1 实验目的、内容与过程

## 一、实验目的：

- (1) 掌握动态规划算法设计思想。
- (2) 掌握鸡蛋坠落问题的动态规划解法。

## 二、实验内容：

动态规划将问题划分为更小的子问题，通过子问题的最优解来重构原问题的最优解。动态规划中的子问题的最优解存储在一些数据结构中，这样我们就不必在再次需要时重新处理它们。任何重复调用相同输入的递归解决方案，我们都可以使用动态规划对其进行优化。

鸡蛋掉落问题是理解动态规划如何实现最佳解决方案的一个很好的例子。问题描述如下：

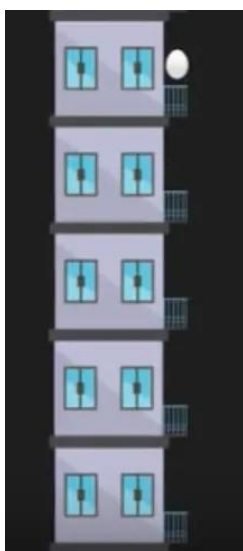


图 1 鸡蛋掉落问题示意图

我们需要用鸡蛋确认在多大的楼层鸡蛋落下来会破碎，这个刚刚使鸡蛋破碎的楼层叫门槛层，门槛楼层是鸡蛋开始破碎的楼层，上面所有楼层的鸡蛋也都破了。另外，如果鸡蛋从门槛楼层以下的任何楼层掉落，它都不会破碎。如上图所示，如果有 5 层，我们只有 1 个鸡蛋，要找到门槛层，则必须尝试从每一层一层一层地放下鸡蛋，从第一层到最后一层，如果门槛层是第  $k$  层，那么鸡蛋就会在第  $k$  层抛下时破裂，应该做了  $k$  次试验。

注意：我们不能随机选择任何楼层，例如，如果我们选择 4 楼并放下鸡蛋并且它打破了，那么它不确定它是否也从 3 楼打破。因此，我们无法找到门槛层，因为鸡蛋一旦破碎，就无法再次使用。

给定建筑物的一定数量的楼层（比如  $f$  层）和一定数量的鸡蛋（比如  $e$  鸡蛋），找出阈值地板必须执行的最少的鸡蛋掉落试验的次数，注意，这里要求的是试验的测试，不是鸡蛋的个数。还要记住的一件事是，我们寻找的是找到门槛层所需的最少鸡蛋掉落试验次数，而不是门槛层下限本身。

问题约束条件：

- 从跌落中幸存下来的鸡蛋可以再次使用。
- 破蛋必须丢弃。
- 摔碎对所有鸡蛋的影响都是一样的。
- 如果一个鸡蛋掉在地上摔碎了，那么它从高处掉下来也会摔碎。
- 如果一个鸡蛋在跌落中幸存下来，那么它在较短的跌落中也能完整保留下来。

#### 四、实验内容及过程：

### 蛮力法分析

我用一个小规模数据的例子来阐述蛮力法是怎么解决这个问题的：

蛮力法解决 **2 个鸡蛋、4 层楼** 时，会把“第一次从哪一层扔”**全部枚举一遍**。程序从  $dp(2,4)$  开始，依次尝试先扔 1 楼、2 楼、3 楼、4 楼。每尝试一种方案，都要考虑两种结果：如果鸡蛋碎了，就说明临界楼层在当前楼层下面，鸡蛋数量减少，问题变成  $dp(K-1, i-1)$ ；如果鸡蛋没碎，就说明临界楼层在当前楼层上面，鸡蛋数量不变，问题变成  $dp(K, N-i)$ 。因为我们要保证在最坏情况下也能找到答案，所以每一种扔法的代价是： $1 + \max(\text{碎了后的次数}, \text{没碎后的次数})$ 。

伪代码如下：

```
def BruteForce(K, N):
```

```
    如果 N == 0:
```

```
        返回 0
```

```
    如果 K == 1:
```

```
        返回 N
```

```
    ans = 无穷大
```

```
    对 i 从 1 到 N:
```

```
        碎了 = BruteForce(K - 1, i - 1)
```

```
        没碎 = BruteForce(K, N - i)
```

```
        当前最坏次数 = 1 + max(碎了, 没碎)
```

```
        ans = min(ans, 当前最坏次数)
```

```
    return ans
```

但是，蛮力法有一个很明显的缺陷：他会进行大量的重复运算。我在下面用两个树形图分别介绍了“第一次选择从 2F 扔”和“第一次选择从 1F 扔”的流程。

可以看见，仅仅只是**超小数据规模**的**两个分支**选择，就出现了两个重复的节点。不敢想象数据达到较大规模时的算力浪费情况了！

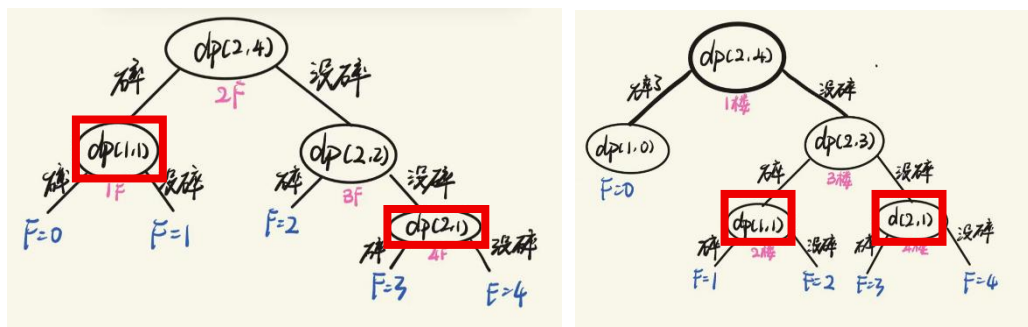


图 2 （左）第一次从 2F 投掷的递归分支示意图

图 3 （右）第一次从 1F 投掷的递归分支示意图

## 动态规划分析

以 **2 个鸡蛋、4 层楼** 为例，动态规划会把“大问题”拆成很多“小问题”来算。我们定义  $dp(K, N)$  表示：有  $K$  个鸡蛋、面对  $N$  层楼时，在最坏情况下最少需要扔几次。比如  $dp(2, 4)$  不是直接猜答案，而是尝试第一步从第 1、2、3、4 层分别扔。每次扔都会出现两种情况：鸡蛋碎了，就变成  $dp(1, i-1)$ ；鸡蛋没碎，就变成  $dp(2, 4-i)$ 。因为我们要保证最坏情况也能测出来，所以每个选择都取两种情况里的较大值，再加上当前这一次扔蛋。它的核心就是：**先算并保存小规模楼层/鸡蛋数的结果，再用这些结果推出更大问题，避免重复计算。**

动态规划中，最重要的就是下面这个递推式：

$$dp(K, N) = 1 + \min_{1 \leq i \leq N} (\max(dp(K-1, i-1), dp(K, N-i)))$$

我们以 3 个蛋，5 层楼为例子：

初始时表格为：

表 1-三个鸡蛋五层楼的数据表格

| 鸡蛋数 \ 楼层数 | 0 | 1 | 2 | 3 | 4 | 5 |
|-----------|---|---|---|---|---|---|
| 1 个蛋      | 0 | 1 | 2 | 3 | 4 | 5 |
| 2 个蛋      | 0 | 1 | ? | ? | ? | ? |
| 3 个蛋      | 0 | 1 | ? | ? | ? | ? |

以  $dp(2, 2)$  为例，我们看看要用到哪些已知数据算出来：

表 2-三个鸡蛋五层楼能计算的初始数据

| 第一次扔 | 碎了           | 没碎           | 最坏次数             |
|------|--------------|--------------|------------------|
| 1 楼  | $dp(1, 0)=0$ | $dp(2, 1)=1$ | $1+\max(0, 1)=2$ |
| 2 楼  | $dp(1, 1)=1$ | $dp(2, 0)=0$ | $1+\max(1, 0)=2$ |

则  $dp(2, 2)=\min(2, 2)=2$

因此，可以看出：填  $dp[x][y]$  需要看：

上一行： $dp[x-1][0], dp[x-1][1], \dots, dp[x-1][y-1]$ ；

当前行： $dp[x][0], dp[x][1], \dots, dp[x][y-1]$

以此类推，我们可以填写完整的表格：

表 3-三个鸡蛋五层楼完整表格

| 鸡蛋数 \ 楼层数 | 0 | 1 | 2 | 3 | 4 | 5 |
|-----------|---|---|---|---|---|---|
| 1 个蛋      | 0 | 1 | 2 | 3 | 4 | 5 |
| 2 个蛋      | 0 | 1 | 2 | 2 | 3 | 3 |
| 3 个蛋      | 0 | 1 | 2 | 2 | 3 | 3 |

## 动态规划 + 二分查找

假如我们有 6 个鸡蛋，100 层楼。

对于普通的动态规划来说，他要枚举“第一次从 1 楼扔”，“第一次从 2 楼扔”，……，“第一次从 100 楼扔”。假如第一次选择了从第一层扔，鸡蛋没碎，那就要计算  $dp(6, 99)$ ，然后对于  $dp(6, 99)$ ，我们又要开始枚举“第一次从 1 楼扔”，“第

一次从 2 楼扔”，……，“第一次从 99 楼扔”。别忘了，我们有 100 条这样的分支（100 层楼），有 100 条这么长长的线。

如果我们直接暴力转移求解每个状态的  $dp$  值，时间复杂度是为  $O(kn^2)$ ，即一共有  $O(kn)$  个状态，对于每个状态枚举扔鸡蛋的楼层  $x$ ，需要  $O(n)$  的时间。因此我们需要想办法优化枚举的时间复杂度。

$$dp(k, n) = 1 + \min_{1 \leq x \leq n} \max(dp(k-1, x-1), dp(k, n-x))$$

在这个公式中，有一个很重要的信息，再固定鸡蛋数目的情况下， $dp$  是一个随着楼层数变大而单调递增的单调函数（ $dp(2, 3) = 2$ ， $dp(2, 4) = 3$ ， $dp(2, 5) = 3$ ， $dp(2, 6) = 3$ ， $dp(2, 7) = 4$ ）。

但是随着鸡蛋数目的增加，问题不一定会变得更难，因为多出来的鸡蛋可以选择不用，所以最差也不会比原来更差。

再回到公式中，我们要求的是：

$$\max(dp(k-1, x-1), dp(k, n-x))$$

整体可能会是这样：

当  $x$  很小时  $dp(k-1, x-1)$  很小， $dp(k, n-x)$  很大。最坏情况由  $dp(k, n-x)$  决定。

当  $x$  很大时  $dp(k-1, x-1)$  很大， $dp(k, n-x)$  很小。最坏情况由  $dp(k-1, x-1)$  决定。

如果是  $x$  处于中间某处，这两个式子基本平和，最坏情况也不会变得很坏。

最优解一定出现在二者接近平衡的位置附近，所以可以用二分查找优化原本的线性枚举。

```
class Solution {
    unordered_map<int, int> memo;
    int dp(int k, int n) {
        if (memo.find(n * 100 + k) == memo.end()) {
            int ans;
            if (n == 0) {
                ans = 0;
            } else if (k == 1) {
                ans = n;
            } else {
                int lo = 1, hi = n;
                while (lo + 1 < hi) {
                    int x = (lo + hi) / 2;
                    int t1 = dp(k - 1, x - 1);
                    int t2 = dp(k, n - x);

                    if (t1 < t2) {
                        lo = x;
                    } else if (t1 > t2) {
                        hi = x;
                    } else {
                        lo = hi = x;
                    }
                }
            }
        }
    }
}
```

```

        ans = 1 + min(max(dp(k - 1, lo - 1), dp(k, n - lo)),
                      max(dp(k - 1, hi - 1), dp(k, n - hi)));
    }

    memo[n * 100 + k] = ans;
}

return memo[n * 100 + k];
}
public:
    int superEggDrop(int k, int n) {
        return dp(k, n);
    }
};

```

## 数学法

在前两种方法中，我们通常从“给定  $k$  个鸡蛋和  $n$  层楼，最少需要尝试多少次”这一角度出发进行分析。普通动态规划需要枚举每一次扔鸡蛋的楼层，并比较鸡蛋碎与不碎两种情况下的最坏结果，因此状态转移过程较为复杂。

方法三采用一种逆向思维：不直接求“ $n$  层楼最少需要多少次”，而是反过来思考：如果最多只能尝试  $t$  次，并且有  $k$  个鸡蛋，那么最多可以确定多少层楼的临界楼层？

设  $f(t, k)$  表示在最多尝试  $t$  次、拥有  $k$  个鸡蛋，最多能够确定答案的楼层数。

只要算出  $f(t, k) \geq n$ ，这个  $t$  就是我们要找到的答案。

对于状态  $f(t, k)$ ，考虑当前扔出一个鸡蛋后可能出现的两种情况：

第一种情况是鸡蛋碎了。此时说明临界楼层在当前楼层的下方，并且鸡蛋数量减少一个，剩余尝试次数也减少一次。因此，当前楼层下方最多可以继续确定： $f(t - 1, k - 1)$

第二种情况是鸡蛋没有碎。此时说明临界楼层在当前楼层的上方，鸡蛋数量不变，但剩余尝试次数减少一次。因此，当前楼层上方最多可以继续确定： $f(t - 1, k)$ 层。

再加上当前扔鸡蛋的这一层本身，可以得到状态转移方程：

$$f(t, k) = 1 + f(t - 1, k - 1) + f(t - 1, k)$$

因此，该方法可以看作是一种基于数学归纳思想的动态规划。先确定最简单的边界状态，再利用递推公式逐步推出更大规模的状态。与普通动态规划不同的是，该方法不需要枚举“从第几层开始扔”，而是直接计算在一定尝试次数和鸡蛋数量下的最大覆盖楼层数。当覆盖楼层数第一次大于或等于目标楼层数  $n$  时，即可得到最少尝试次数。

表 4-鸡蛋能覆盖楼层数的举例

| 尝试次数 $t$ | 1 个鸡蛋最多覆盖楼层 | 2 个鸡蛋最多覆盖楼层 |
|----------|-------------|-------------|
| 1        | 1           | 1           |
| 2        | 2           | 3           |
| 3        | 3           | 6           |

|   |   |    |
|---|---|----|
| 4 | 4 | 10 |
| 5 | 5 | 15 |

如果问题是 2 个鸡蛋、10 层楼，则由表可知，最少需要 4 次尝试。

```
class Solution {
public:
    int superEggDrop(int k, int n) {
        if (n == 1) {
            return 1;
        }
        vector<vector<int>> f(n + 1, vector<int>(k + 1));
        for (int i = 1; i <= k; ++i) {
            f[1][i] = 1;
        }
        int ans = -1;
        for (int i = 2; i <= n; ++i) {
            for (int j = 1; j <= k; ++j) {
                f[i][j] = 1 + f[i - 1][j - 1] + f[i - 1][j];
            }
            if (f[i][k] >= n) {
                ans = i;
                break;
            }
        }
        return ans;
    }
}
```

## 动态规划 + 二分 + 数学法

在数学法中，我们已经把问题从“有  $k$  个鸡蛋、 $n$  层楼最少扔几次”转换成“给定  $t$  次尝试、 $k$  个鸡蛋最多能覆盖多少层楼”。如果  $f(t, k) \geq n$ ，说明  $t$  次已经够用。

这个判断具有单调性：当尝试次数  $t$  增加时，可覆盖楼层数一定不会减少。因此答案可以在尝试次数  $t$  上做二分查找，而不是像数学法那样从 1 次、2 次、3 次逐次往上试。

判断某个  $t$  是否可行时，可以继续使用动态规划递推  $f(t, k) = f(t-1, k-1) + f(t-1, k) + 1$ ；也可以利用它对应的组合数形式  $f(t, k) = C(t, 1) + C(t, 2) + \dots + C(t, k)$ 。实验中的“动态规划 + 二分 + 数学法”采用后一种判断方式，并在覆盖楼层数已经超过  $n$  时提前停止，避免大数继续膨胀。

以 2 个鸡蛋、100 层楼为例，普通数学法要从  $t=1$  开始逐次计算覆盖层数；该方法先判断中间次数是否足够，若足够就向更小次数寻找，否则向更大次数寻找，最后得到最小可行  $t$ 。因此它特别适合  $10^{21}$ 、 $10^{24}$  甚至更大楼层数的测试。

伪代码如下：

```
function CanReach(t, k, n):
    covered = 0
```

```
combination = 1
for i from 1 to min(k, t):
    combination = combination * (t - i + 1) / i
    covered = covered + combination
    if covered >= n:
        return true
return false

function BinaryMathEggDrop(k, n):
    low = 0, high = 1
    while CanReach(high, k, n) == false:
        high = high * 2

    while low < high:
        mid = (low + high) / 2
        if CanReach(mid, k, n):
            high = mid
        else:
            low = mid + 1
    return low
```

该方法中，每次可行性判断最多计算  $k$  个组合数项，二分查找次数为  $O(\log N)$ ，所以理论时间复杂度约为  $O(K \log N)$ ，空间复杂度为  $O(1)$ ；如果使用高精度整数，还需要额外考虑大整数位数带来的常数开销。

## 理论时间复杂度比较

为了和实验结果对应，这里用  $K$  表示鸡蛋数， $N$  表示楼层数， $T$  表示最终求出的最少尝试次数。各算法的理论时间复杂度对比如下：

表 5-各方法时间复杂度比较

| 算法      | 理论时间复杂度         | 复杂度来源                                  | 适用特点                 |
|---------|-----------------|----------------------------------------|----------------------|
| 蛮力法     | 指数级, 约 $O(2^n)$ | 递归枚举首次投掷楼层，且不保存重复子问题，递归树会迅速膨胀。         | 只适合小规模样例和原理说明。       |
| 普通动态规划  | $O(Kn^2)$       | 共有 $O(KN)$ 个状态，每个状态还要枚举 1 到 $N$ 的投掷楼层。 | 比蛮力法稳定，但楼层数大时增长很快。   |
| 动态规划+二分 | $O(KN \log N)$  | 状态数仍为 $O(KN)$ ，但每个状态用二分查找代替线性枚举。       | 适合中大规模，仍需要按楼层建表。     |
| 数学法     | $O(KT)$         | 按尝试次数递推覆盖楼层数，每增加一次尝试更新 $K$ 个鸡蛋状态。      | 当 $T$ 远小于 $N$ 时效率很高。 |



|             |               |                                  |                     |
|-------------|---------------|----------------------------------|---------------------|
| 动态规划+二分+数学法 | $O(K \log N)$ | 在尝试次数上二分，每次用组合数形式快速判断 $t$ 次是否足够。 | 最适合超大楼层数场景。         |
| 空间优化        | $O(KT)$       | 时间与数学法相同，只是把二维覆盖表压缩成一维数组。        | 主要优势是把空间降为 $O(K)$ 。 |

。蛮力法到普通 DP 解决的是重复计算问题；DP+二分解决的是单个状态枚举过慢的问题；数学法和二分数学法则进一步改变观察角度，把楼层枚举转化为覆盖能力判断，因此在极限数据规模下优势最明显。

## 空间优化

当我们使用了数学法以后，空间上可以将原先的二维数组降为一维数组。  
假如我们有 3 个蛋，10 层楼，原先的二维数组长这样：

表 6-二维数组储存数据

| t / k | 1 个蛋 | 2 个蛋 | 3 个蛋 |
|-------|------|------|------|
| 1     | 1    | 1    | 1    |
| 2     | 2    | 3    | 3    |
| 3     | 3    | 6    | 7    |
| 4     | 4    | 10   | ?    |

当我们计算 3 个蛋 4 次尝试时，代入公式可以得到：

$$f(4, 3) = 1 + f(3, 2) + f(3, 3)$$

我们只需要上一轮尝试的两个数据就可以得出。所以其实如果用二维数组来存放就会产生浪费（我们根本不需要比较早期的数据）。

因此我们可以将存放的数据设置为一个 dp 一维数组,dp[j]表示当前尝试轮数下，j 个鸡蛋能测几层楼：

第 1 次尝试

$$dp = [0, 1, 1, 1]$$

第 2 次尝试

$$dp = [0, 2, 3, 3]$$

第 3 次尝试

$$dp = [0, 3, 6, 7]$$

第 4 次尝试

$$dp = [0, 4, 10, 14]$$

注意：每一次都是从右往左更新，右边较大的数据会用到较小的旧数据，提前将较小数据覆盖就无法计算出后面的数据了。

总结：我们只需要一个数组，通过一轮轮尝试不断更新迭代，就可以找到最后的答案。极大的节省了空间资源。

五、实验结果及分析：

实验一：测试算法正确性

表 7-用蛮力法验证动态优化的正确性

| 序号 | 鸡蛋/层数 |    | 最优尝试次数 | 蛮力法<br>(ms) | 动态规划<br>(ms) | 校验 |
|----|-------|----|--------|-------------|--------------|----|
| 1  | 1     | 3  | 3      | 0.0005      | 0.0043       | 通过 |
| 2  | 1     | 4  | 4      | 0.0004      | 0.0030       | 通过 |
| 3  | 1     | 6  | 6      | 0.0005      | 0.0023       | 通过 |
| 4  | 1     | 10 | 10     | 0.0003      | 0.0021       | 通过 |
| 5  | 2     | 1  | 1      | 0.0004      | 0.0024       | 通过 |
| 6  | 2     | 4  | 3      | 0.0835      | 0.1562       | 通过 |
| 7  | 2     | 6  | 3      | 0.0075      | 0.0046       | 通过 |
| 8  | 2     | 8  | 4      | 0.0484      | 0.0314       | 通过 |
| 9  | 3     | 1  | 1      | 0.0004      | 0.0029       | 通过 |
| 10 | 3     | 2  | 2      | 0.0006      | 0.0036       | 通过 |
| 11 | 3     | 5  | 3      | 0.0023      | 0.0057       | 通过 |
| 12 | 3     | 7  | 3      | 0.0119      | 0.0087       | 通过 |
| 13 | 3     | 8  | 4      | 0.0309      | 0.0102       | 通过 |
| 14 | 3     | 9  | 4      | 0.0913      | 0.0333       | 通过 |
| 15 | 4     | 4  | 3      | 0.0013      | 0.0059       | 通过 |
| 16 | 4     | 5  | 3      | 0.0035      | 0.0118       | 通过 |
| 17 | 4     | 6  | 3      | 0.0079      | 0.0094       | 通过 |
| 18 | 4     | 7  | 3      | 0.0212      | 0.0116       | 通过 |
| 19 | 4     | 9  | 4      | 0.6395      | 0.0384       | 通过 |
| 20 | 4     | 10 | 4      | 0.5003      | 0.0234       | 通过 |

理论分析：

该表用于验证算法结果是否正确。小规模样例覆盖了 1 到 4 个鸡蛋、1 到 10 楼层的多种组合，其中 1 个鸡蛋时最优尝试次数等于楼层数，符合只能逐层测试的边界条件；当鸡蛋数量增加时，最优尝试次数整体不会增大，说明动态规划结果满足问题的单调性。

表中所有样例的校验结果均为“通过”，说明蛮力法和动态规划法在相同输入下得到的最优尝试次数一致，从实验上验证了状态转移方程  $1 + \max(\text{碎了后的子问题}, \text{没碎后的子问题})$  的正确性。少量小规模数据中蛮力法耗时看起来接近甚至低于动态规划，主要是因为测试规模很小、计时精度和函数开销占比高，不能代表算法在大规模情况下的实际增长趋势。

实验二：不同数据规模下不同算法的效率测试

表 8-不同算法的效率测试

| 数据规模                      | 方法          | 所用时间/ms  | 是否完成 |
|---------------------------|-------------|----------|------|
| 4 个鸡蛋，20 层楼               | 蛮力法         | 742.626  | 是    |
|                           | DP          | 0.191    | 是    |
|                           | DP+二分       | 0.031    | 是    |
|                           | DP+数学法优化    | 0.008    | 是    |
|                           | DP+二分+数学法优化 | 0.004    | 是    |
| 12 个鸡蛋，2000 层楼            | DP          | 58.061   | 是    |
|                           | DP+二分       | 1.159    | 是    |
|                           | DP+数学法优化    | 0.015    | 是    |
|                           | DP+二分+数学法优化 | 0.007    | 是    |
|                           | DP          | 2806.987 | 是    |
| 20 个鸡蛋，10000 层楼           | DP+二分       | 14.265   | 是    |
|                           | DP+数学法优化    | 0.035    | 是    |
|                           | DP+二分+数学法优化 | 0.007    | 是    |
|                           | DP+二分       | 707      | 是    |
|                           | DP+数学法优化    | 0.033    | 是    |
| 40 个鸡蛋，200000 层楼          | DP+二分+数学法优化 | 0.01     | 是    |
|                           | DP+二分       | 2364.031 | 是    |
|                           | DP+数学法优化    | 7.265    | 是    |
|                           | DP+二分+数学法优化 | 0.029    | 是    |
|                           | DP+数学法优化    | 0.099    | 是    |
| 50 个鸡蛋，500000 层楼          | DP+二分+数学法优化 | 0.018    | 是    |
|                           | DP+二分       | 16.273   | 是    |
|                           | DP+数学法优化    | 0.034    | 是    |
|                           | DP+二分+数学法优化 | 9.42     | 是    |
|                           | DP+数学法优化    | 0.036    | 是    |
| 60 个鸡蛋，1000000 层楼         | DP+二分+数学法优化 | 0.04     | 是    |
|                           | DP+二分       |          |      |
|                           | DP+数学法优化    |          |      |
|                           | DP+二分+数学法优化 |          |      |
|                           | DP+数学法优化    |          |      |
| 5 个鸡蛋，10 <sup>21</sup> 层楼 | DP+二分+数学法优化 |          |      |
|                           | DP+二分       |          |      |
|                           | DP+数学法优化    |          |      |
|                           | DP+二分+数学法优化 |          |      |
|                           | DP+数学法优化    |          |      |
| 6 个鸡蛋，10 <sup>24</sup> 层楼 | DP+二分+数学法优化 |          |      |
|                           | DP+二分       |          |      |
|                           | DP+数学法优化    |          |      |
|                           | DP+二分+数学法优化 |          |      |
|                           | DP+数学法优化    |          |      |
| 2 个鸡蛋，10 <sup>30</sup> 层楼 | DP+二分+数学法优化 |          |      |
|                           | DP+二分       |          |      |
|                           | DP+数学法优化    |          |      |
|                           | DP+二分+数学法优化 |          |      |
|                           | DP+数学法优化    |          |      |



图 4 不同算法耗时走势对比图

理论分析:

该表体现了不同优化方法在数据规模扩大时的时间复杂度差异。蛮力法需要递归枚举每一次可能的投掷楼层，并且会反复计算大量相同子问题，所以在 4 个鸡蛋、20 层楼时已经达到 742.626 ms，扩展能力较弱。普通 DP 通过保存子问题结果避免重复计算，但每个状态仍需枚举楼层，理论复杂度约为  $O(KN^2)$ ，因此在 20 个鸡蛋、10000 层楼时耗时达到 2806.987 ms。

DP+二分利用“鸡蛋碎”和“鸡蛋没碎”两个子问题随投掷楼层变化呈相反单调趋势这一性质，把每个状态中的线性枚举优化为二分查找，因此在相同规模下明显快于普通 DP。DP+数学法优化则改变状态定义，计算  $t$  次尝试、 $k$  个鸡蛋最多能覆盖的楼层数，递推式为  $f(t, k) = f(t-1, k-1) + f(t-1, k) + 1$ ，不再直接依赖从 1 到  $N$  的逐层枚举，所以可以处理  $10^{21}$ 、 $10^{24}$  级别的楼层数。进一步结合二分思想后，只需快速判断给定尝试次数能否覆盖目标楼层，运行时间继续下降，表中大规模样例仍能保持在极短时间内完成。

表 9-各算法的阈值

| 方法       | 极限数据规模                 | 极限运行时间(ms) | 极限答案  |
|----------|------------------------|------------|-------|
| 蛮力法      | S06: 4 个鸡蛋, 20 层楼      | 742.626    | 5     |
| DP       | S13: 20 个鸡蛋, 10000 层楼  | 2806.987   | 14    |
| DP+二分    | S17: 50 个鸡蛋, 500000 层楼 | 2364.031   | 19    |
| DP+数学法优化 | S26: 6 个鸡蛋, 10^24 层楼   | 9.42       | 29940 |

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |                         |      |                      |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------|------|----------------------|
| DP+二分+数学法优化                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        | S27: 2 个鸡蛋,<br>10^30 层楼 | 0.04 | 1414213562373<br>095 |
| <b>理论分析:</b> <p>极限数据规模表进一步说明了各算法的瓶颈位置。蛮力法的极限停留在 4 个鸡蛋、20 层楼，原因是递归树随楼层数增长迅速膨胀；普通 DP 虽然能够推进到 20 个鸡蛋、10000 层楼，但状态数量和楼层枚举次数仍然随 N 快速增加。DP+二分把枚举过程压缩后，可扩展到 50 个鸡蛋、500000 层楼，但本质上仍然要围绕楼层规模建立状态。</p> <p>数学法和二分数学优化的极限明显更高，是因为它们关注“给定尝试次数能覆盖多少层”，而不是“每一层第一次扔要怎么选”。当鸡蛋数固定时，覆盖楼层数会随着尝试次数按组合数规律增长，因此即使目标楼层达到 <math>10^{24}</math> 或 <math>10^{30}</math>，也可以通过较少轮次的递推或二分判断得到答案。例如 2 个鸡蛋时覆盖能力与三角数有关，最少尝试次数约随 <math>\sqrt{N}</math> 增长，所以 <math>10^{30}</math> 层楼虽然答案很大，但计算过程仍然可以非常快。</p>                        |                         |      |                      |
| <b>六、实验结论:</b> <p>本次实验围绕鸡蛋掉落问题，对蛮力法、普通动态规划、动态规划加二分优化、数学递推法、动态规划+二分+数学法优化以及空间优化方法进行了实现和对比。实验结果表明，所有小规模样例均能得到一致的最优尝试次数，说明动态规划状态定义、边界条件和状态转移方程是正确的。</p> <p>从效率上看，蛮力法适合帮助理解问题结构，但由于重复子问题过多，实际可处理规模很小；普通动态规划通过记忆化或表格化保存结果，显著减少了重复计算，但在大规模楼层数下仍会受到状态数量和枚举过程的限制。二分优化利用单调性降低了单个状态的求解代价，而数学法通过反向思考“若干次尝试最多能确定多少层楼”，把问题转化为覆盖能力递推，使算法在超大规模数据上仍能快速求解。</p> <p>因此，本实验不仅验证了动态规划“拆分子问题、保存子问题最优解、由小到大推出整体最优解”的核心思想，也说明了算法优化应从状态定义、转移方式和问题单调性等角度入手。对于鸡蛋掉落问题，小规模场景可用普通 DP 保证直观性，而面对大规模楼层数时，数学递推及其二分优化方法更具有实际应用价值。</p> |                         |      |                      |

